CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 2

CO2: Analyze software requirements to design scalable and efficient system architectures using collaborative tools like Jira and Confluence, along with structured modeling techniques such as UML diagrams and architectural design patterns.

Assignment Title: Design and Validate Requirements and Architecture for the Omnichannel Order Fulfilment Platform

Nature of Assignment: Individual

Duration: One week

Total Marks: 50

Submission Date: _____

Scenario — Order Fulfilment Requirements and Scalable Architecture

A retailer needs an order fulfilment architecture that connects checkout, stock reservation, payment callbacks, warehouse picking, courier tracking and refunds without losing consistency or exposing restricted customer information. Repeated payment callbacks, two buyers competing for the last unit and failed courier synchronization create conflicting requirements for speed, availability, accountability and confidentiality. Analyze stakeholder needs and create a testable requirements specification with traceable UML models and justified architectural alternatives. Validate order and refund invariants, data ownership and failure handling. Redesign the selected architecture for additional regions, increased demand and disconnected warehouse scanners while explaining consistency, availability and operating-cost trade-offs.

The retailer proposes an omnichannel order fulfilment platform. Its intended modules are a storefront catalogue, cart and checkout, payment integration, inventory synchronization, order tracking, a warehouse picking dashboard, returns and refunds, a support console and sales analytics. The system should connect a customer order to stock reservation, payment confirmation, picking, dispatch, tracking, delivery and, where needed, an authorized return and refund.

Architecture conditions for the representative release are stated below.

Observation	Scenario planning condition
Operational scope	Checkout, stock reservation, payment confirmation, picking, dispatch, tracking, returns and refunds
Users	Administrator, picker, support agent, finance reviewer, store manager and customer

Reference data	Synthetic products, orders and stock movements; no real customer or card data
Integration constraint	Payment gateway or courier API may be unavailable or may send duplicate callbacks; preserve local order records and retry safely
Validation need	Verify stock, order status, price, authorization, persistence and refund audit behaviour

Principal users are enterprise administrators, warehouse pickers, support agents, finance reviewers, store managers and customers. They need different permissions. Pickers require accurate pick lists; support agents require readable order and return status; finance reviewers require auditable refund approvals; engineers require traceable links between orders, payments and courier shipments. Automation must preserve human responsibility for refund approval and stock corrections.

Customers want fast checkout and clear tracking; pickers want reliable stock reservation; support agents want order visibility without seeing payment data; finance reviewers require approval before refunds above a limit. Management wants consolidated sales, stock and refund reporting. These interests may conflict when speed, privacy and refund controls compete.

Use a manageable representative release and synthetic orders and stock events. Jira, Confluence, GitHub, Jenkins and Docker are suitable reference tools; select and justify alternatives where appropriate. A React interface, FastAPI service and PostgreSQL database are one feasible reference architecture, not compulsory choices. External integrations may be demonstrated through controlled test interfaces when live access is unavailable, with their status disclosed.

Define workload, service-level policies and acceptance thresholds as explicit planning assumptions. Dashboard values, stock counts and screenshots are illustrative evidence of displayed behaviour, not proof of production accuracy, performance, live inventory control or secure payment handling. Verify claims through repeatable checks. Distinguish deterministic rules, such as fixed low-stock thresholds, from trained machine-learning forecasting, and label simulations and proposed extensions accurately.

Design Challenge:

Repeated payment callbacks may create duplicate payments or orders, while two customers may buy the last unit. Courier synchronization may fail after the order is stored. A refund must not bypass its required approval, and a support agent must not gain finance privileges merely by selecting another role in the interface.

The organization requires consistent order state, auditable refunds and dependable integration while its small technical team initially operates a modest deployment. Additional services must be justified against operational cost and complexity.

Design and validate a requirements specification and scalable system architecture using collaborative and modeling tools of your choice that address these conditions.

Develop alternative architectural solutions and create a justified design connecting stakeholder needs, allocation rules, quality requirements and architectural decisions. Maintain traceability between requirements, UML models, architectural components and validation scenarios.

Examine at least two suitable tool arrangements for requirements tracking, collaborative documentation and UML modeling. Select a tool or combination of tools and justify its suitability, considering traceability, collaboration, version history, diagram support, accessibility and cost. Explain any limitations and how you will address them.

Validate the design against deliberately constructed scenarios. Use the findings to revise your requirements and architecture, and defend the final design's consistency, scalability and limitations. Explain how the selected tools support collaboration and keep the specification, models and design decisions consistent.

Q1 – Investigate Stakeholder Needs and Resolve Conflicts 5 Marks

Investigate the needs of customers, pickers, support agents, administrators, store managers and finance reviewers.

- Formulate at least three questions that investigate an unresolved requirement or stakeholder conflict.
- Gather findings through a small stakeholder discussion, questionnaire or process review. Where access is unavailable, use clearly labelled simulated responses.
- Identify at least three conflicting expectations.
- Develop and justify a resolution for each conflict.
- Define the system boundary, principal actors and scope exclusions.

Do not assume that every stakeholder request should become a requirement.

Evidence Expected

Investigation questions, findings or labelled simulated responses, a stakeholder analysis, conflict-resolution decisions and a justified system boundary.

Q2 – Create a Prioritized and Testable Requirements Specification 8 Marks

Create a requirements specification in a selected collaborative documentation tool and represent prioritized requirements in a selected issue tracker. Jira and Confluence are reference options; justify the arrangement chosen.

- At least twelve functional requirements and six non-functional requirements.
- Unique identifiers, sources, priorities and acceptance criteria.
- Order rules covering price and tax calculation, stock reservation, ownership, permitted status changes, dispatch deadlines, duplicate payment callbacks and refund approvals.
- Measurable quality requirements for performance, availability, security and usability.
- Explicit assumptions and unresolved questions.
- A justified prioritization method.

- Replacement of vague statements such as “checkout should be fast” with requirements that specify the operating condition and an observable acceptance threshold.
- An explanation of how at least two stakeholder conflicts influenced the final specification.

Evidence Expected

Collaborative specification, requirement records, measurable acceptance criteria, prioritization rationale and documented assumptions.

Q3 – Construct UML Models and Maintain Traceability 7 Marks

Create a consistent set of UML models covering:

- A use case diagram showing actors and principal interactions.
- A class diagram representing important domain concepts and relationships.
- A sequence diagram tracing a payment callback through order confirmation, shipment creation and courier synchronization, including a retry or failure.
- A state machine for Placed, Paid, Packed, Dispatched, Delivered and Closed orders, including authorized cancellation or return if supported.
- Approval, rejection, execution and rollback behaviour for refunds, modelled where relevant.
- A traceability matrix connecting Requirement → UML Element → Architectural Component → Validation Scenario.
- Identification and resolution of at least two inconsistencies discovered while connecting these representations.

Evidence Expected

Four UML diagrams, a traceability matrix, identified inconsistencies and documented corrections.

Q4 – Design and Justify Alternative Architectures 8 Marks

Develop two meaningfully different architectural alternatives for the system. Select appropriate patterns, such as a modular monolith, microservices or event-driven architecture. For each alternative:

- Define the principal components and responsibilities.
- Explain communication and data ownership.
- Describe how duplicate payment processing and overselling of the last unit would be prevented.
- Examine scalability, consistency, availability, security and operational complexity.
- Identify likely failure points and maintenance implications.

Then select an architecture and create component and deployment diagrams. Document at least three architectural decisions in your selected documentation tool. State each problem, alternatives, selection, justification and consequences.

Evidence Expected

Two architectural alternatives, a reasoned comparison, selected component and deployment diagrams, and three architectural decision records.

Q5 – Validate Critical Behaviour and Revise the Design 7 Marks

Define order and refund rules that must remain true throughout operation. Include invariants such as:

The same gateway event identity must not create multiple payments or order confirmations; an unapproved refund must not be marked as executed through an approval-gated operation.

Specify preconditions and postconditions for at least two critical operations. Construct at least six validation scenarios, including:

- Two customers attempting to buy the last unit of a product.
- A repeated payment callback arriving while the original callback is being processed.
- An approved refund failing at the gateway and requiring a controlled retry or rollback.
- A dispatch attempt without confirmed payment or pick confirmation.
- A courier synchronization request failing and then being retried.
- A customer or support agent attempting to approve a refund or access another customer's order.

Before validation, predict the expected behaviour and identify which requirement or design decision should enforce it.

Validate using a small prototype, executable model, structured walkthrough or another justified method. Clearly distinguish demonstrated results from predictions.

Identify a weakness and revise the relevant requirements, UML models or architecture.

Evidence Expected

Invariants, operation contracts, six validation scenarios, expected and observed behaviour, validation evidence and a documented design revision.

Q6 – Redesign for Expansion and Partial Failure 10 Marks

Changed Requirement

The organization expands from three to twelve regions. The planning workload increases from 50 to 500 simultaneous users and from 100 to 1,000 stock, payment and scan events per minute. These are design assumptions rather than demonstrated system capacity.

- Each region can define local tax, shipping and refund-approval rules.
- A disconnected warehouse scanner node must buffer uniquely identified scan and stock events for later synchronization.
- A locally queued refund approval must not be displayed as globally approved before server validation.
- Management requires consolidated sales, stock and refund reports.

- The operating budget permits only a limited increase in infrastructure expenditure.

Create a revised requirements and architecture design that addresses the expansion.

Component	Required Work	Marks
A. Diagnose the impact	Identify affected requirements, assumptions, components and quality attributes.	2
B. Create the revised architecture	Redesign component responsibilities, data ownership, communication and deployment. Explain pending buffered events, deduplication and server-validated approval.	4
C. Validate the redesign	Construct at least three additional scenarios covering increased load, synchronization and partial failure. Provide validation evidence appropriate to your method.	2
D. Defend the design	Explain benefits, costs, consistency and availability trade-offs, and remaining limitations.	2

Do not assume that increasing the number of services automatically improves scalability. Connect each architectural change to an identified requirement or limitation.

Evidence Expected

Impact analysis, revised requirements and diagrams, synchronization rules, validation findings and an evidence-supported architectural defence.

Q7 – Individual Technical Reflection 5 Marks

Explain:

- Which requirement had the greatest influence on your architecture.
- Which stakeholder conflict was most difficult to resolve.
- Which inconsistency was discovered through modeling or validation.
- Which architectural trade-off remains unresolved or significant.
- What additional evidence would be needed before implementing the system.

Evidence Expected

A concise reflection demonstrating understanding of the created design and its limitations. Refer to your own specification, diagrams, decisions and validation findings.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Stakeholder investigation and conflict resolution	5
Q2	Prioritized and testable requirements specification	8
Q3	UML modeling and requirements traceability	7
Q4	Alternative architectures and justified design	8
Q5	Critical-behaviour validation and design revision	7
Q6	Day-4 Design Challenge – architecture redesign for expansion and partial failure	10
Q7	Individual technical reflection	5
TOTAL		50

Submission Requirements

- Assignment report covering stakeholder analysis, scope, architectural alternatives and tool selection.
- Collaborative requirements specification and issue-tracker records with unique identifiers and acceptance criteria.
- UML diagrams (use case, class, sequence, state machine, component and deployment) and the traceability matrix.
- Architectural decision records, invariants, operation contracts and validation scenarios with expected and observed behaviour.
- Day-4 revisions: impact analysis, revised requirements and diagrams, synchronization rules and validation findings.
- Individual reflection and acknowledgement of sources and tools used.

OBE Assessment Evidence

Successful completion should demonstrate the ability to analyze conflicting requirements, create a testable specification, model the system with UML, evaluate and select an architecture, validate critical behaviour, and revise the design when scale and failure conditions change.

Assessment Verification

Students may be asked to trace a requirement to its UML element, component and validation scenario, explain an order or refund invariant, justify an architectural decision, or revise the design when a requirement changes.

Overall Assessment Expectation

Investigate → Specify → Model → Architect → Validate → Redesign → Defend

The final submission must demonstrate an original, evidence-supported specification and architecture aligned with CO2, BTL 5 – Evaluate and BTL 6 – Create.